Supply Chain Analytics 2 (2023) 100006

Contents lists available at ScienceDirect

Supply Chain Analytics

journal homepage: www.sciencedirect.com/journal/supply-chain-analytics

A supply chain waste reduction optimization model using beam search algorithms for two-dimensional cutting problems with defects

Mohsen Afsharian

Chair of Production and Logistics, Leibniz FH, University of Applied Sciences, Expo Plaza 11, 30539 Hanover, Germany

ARTICLE INFO

Keywords:
Supply chain
Sustainability
Beam search algorithms
Dynamic programming
NP-hard optimization
Two-dimensional cutting

ABSTRACT

Sustainability in supply chain management addresses various challenges, from waste minimization to resource efficiency maximization. Two-dimensional cutting problems are common problems in most supply chains where small rectangular items need to be cut from large rectangular stock sheets to meet production needs or customer demands. The large stock sheets, produced from materials such as paper, steel, or wood, often contain defects. An optimal cutting solution is needed to avoid overlap with any defects and minimize waste in the cutting process. We propose a supply chain waste reduction optimization model using beam search algorithms for two-dimensional cutting problems with defects. Our proposed solution leverages the power of advanced analytics through a dynamic programming approach. Our algorithms feature variable beam widths and heuristic rules to reduce computation times while yielding high-quality solutions. A simulation model is used to assess the performance of the proposed algorithms.

1. Introduction

In the realm of sustainability in supply chain management, cutting and packing (C&P) problems are of critical importance. They aim to enhance resource efficiency and minimize waste through effective optimization of production and logistics processes. The two-dimensional cutting (TDC) problem, a subcategory of C&P, involves cutting small rectangular items from a large rectangular stock sheet made of materials such as glass, leather, paper, steel, textiles, or wooded board. The objective is to maximize the total value of the small items produced or minimize the waste generated from the cutting process. Different types of TDC problem can be defined by imposing additional constraints, as per the practical requirements of the situation (for an overview, see, e.g., [2,19]). Advanced analytics equipped with methods of combinatorial optimization can also be leveraged to find an optimal solution that takes into consideration sustainability goals, such as waste minimization and resource utilization.

The manufacturing industry often involves cutting small rectangular items from larger rectangular objects that may contain defects. For instance, in the steel industry, steel coils may have contaminated areas that must be avoided when cutting the coils into smaller plates. In many practical scenarios, there is no restriction on the number of pieces of each small item type that can be cut from the large object. Additionally, all cuts must be of the guillotine type and the number of cutting stages is unrestricted. To attain a cost-effective and eco-friendly manufacturing operation, the objective is then to maximize the value of the small items produced by effectively utilizing available resources, minimizing waste through efficient cutting processes, and avoiding the use of materials with defects to ensure high quality standards [3,31]. These efforts will not only enhance the supply chain's performance but also foster sustainability and reduce its environmental impact.

According to Wäscher et al. [32], this TDC problem is classified as "*two-dimensional, rectangular, unconstrained, guillotineable-layout, single large object placement problem with defects*", indicated as 2D_UG_SLOPP_D.[1] The problem at hand is a generalization of the classic Unbounded Single Knapsack Problem, which has been shown to be NP-hard [20]. As such, the 2D_UG_SLOPP_D and its variants belong to the class of NP-hard optimization problems with no efficient method to compute optimal solutions [2]. However, pseudo-polynomial algorithms, such as those based on dynamic programming [3], can solve small to medium sized instances. For larger instances where optimal solutions cannot be efficiently computed, heuristic approaches must be considered to find near-optimal solutions.

Given this background, the dynamic programming algorithms of Afsharian et al. [3] are modified with the incorporation of a beam search strategy [21,28,4]. The proposed algorithms feature variable

E-mail address: mohsen.afsharian@leibniz-fh.de.

[1] Depending on the assumptions, constraints, and the nature of the objectives to follow, several other types of cutting problems with defects with various features and characteristics have been identified, formulated, and studied in the past decades (see, e.g., [14,17,8,10,1,27,29,9,31,26,5,15,22,33]).

https://doi.org/10.1016/j.sca.2023.100006
Received 3 January 2023; Received in revised form 25 February 2023; Accepted 29 March 2023
2949-8635/© 2023 The Author. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

beam widths and various heuristic rules, resulting in significant reduction of computation times while still delivering good solutions. The performance of the proposed algorithms is evaluated through a comprehensive simulation study. The experiments aim to evaluate the capability of the algorithms in delivering efficient and high-quality solutions for realistic-sized problem instances. The results of these simulations provide valuable insights into the strengths and weaknesses of the proposed algorithms and inform further improvements for achieving better performance.

2. Problem definition and the underlying assumptions

Rectangular small items of m different types j ($j = 1, \ldots, m$), each associated with a length l_j^s, a width w_j^s, and a non-negative value v_j, have to be cut from a single rectangular large object of length L_0 and width W_0 such that the total value of the small items resulting from the cutting process is maximized. The following additional assumption are made:

- Edges of small items must be arranged in parallel to the edges of the large object,
- Small items must lie entirely within the large object and must not overlap,
- The demand for each small item type is unlimited,
- For each item type, only one orientation is permitted,
- All cuts must be of the guillotine-type, producing two sub-rectangles,
- The number of stages which are necessary to perform all cuts is not limited,
- The large object contains several defective regions,
- Defects are represented by a set of non-overlapping rectangles of integer dimensions,
- No small item must be assigned to or overlap with any of the defects.

The following definitions and notations are also used in the entire paper:

- A guillotine cut always results in two (sub-) rectangles, which will be called intermediate plates.
- The pair of intermediate plates resulting from a guillotine cut will be called a division.
- In addition to the dimensions, the location of each intermediate plate is indicated by the coordinates of the point where the lower left corner is located on the large object. Let the dimensions of an intermediate plate be (x, y), and let its location on the large object be denoted by (ox, oy), then the plate can be represented by (ox, oy, x, y). Consequently, the large object can be indicated by $(0, 0, L_0, W_0)$.
- A cut is indicated by its position on the corresponding plate: for example, after performing a vertical cut to an intermediate plate $N = (ox, oy, x, y)$ at position z, the two generated intermediate plates will be denoted by (ox, oy, z, y) and $(ox + z, oy, x - z, y)$.
- Let a defect i be of the dimensions (l_i^d, w_i^d), and let its location on the large object be indicated by (x_i^d, y_i^d), then the defect is represented by $(x_i^d, y_i^d, l_i^d, w_i^d)$.
- An intermediate plate $N = (ox, oy, x, y)$ is a pure intermediate plate, if $(x_i^d + l_i^d \leq ox$ or $x_i^d \geq ox + x)$ or $(y_i^d + w_i^d \leq oy$ or $y_i^d \geq oy + y)$ for $i = 1,\ldots,n$. Otherwise, is called contaminated as it either contain or overlaps with a defect.
- The positions of the cuts on the large object are taken from the suggested discretization sets of Afsharian et al. [3], which are in turn based on the discretization sets proposed by Herz [18].

3. Elementary tree-search procedure

There exists a variety of ways for cutting a defective large object, leading to different feasible cutting patterns. The generation of cutting patterns can be represented by means of a tree-search procedure in which all possibilities of cutting the given large object into rectangular small items of different types have to be examined. In addition, several rules can be imposed to the procedure to avoid unnecessary branching. In the following these rules are described and the elementary tree-search procedure is then designed. The following tree-search procedure can be considered as an extension of the tree-search procedures which have been used by Gilmore and Gomory [13], Herz [18] and Beasley [7] already for the corresponding problem without defects.

3.1. Components

Let $\{1,2,\ldots,x - 1\}$ be the set of all possible positions for (guillotine) cuts, which can be carried out to a particular plate $N = (ox, oy, x, y)$ in parallel to the y-axis (vertical cuts), and $\{1,2,\ldots,y - 1\}$ be the set of all possible positions for (guillotine) cuts, which can be carried out to the plate in parallel to the x-axis (horizontal cuts), respectively. In addition, let a 0-cut be an artificial cut by which the plate remains intact. A plate $N = (ox, oy, x, y)$ generated by a 0-cut is called a 0-plate. In addition, a 0-plate is called a final plate if it represents a pure plate, and its dimensions are identical to the dimensions of a small item type, i.e., a small item is obtained.

Throughout the cutting process, one may decide for each particular plate, which can be either an intermediate plate or a 0-plate, whether it can be discarded or not. A particular plate $N = (ox, oy, x, y)$ can be discarded if the following condition holds:

- N is a 0-plate, but not a final plate, i.e., it is either a contaminated plate or its dimensions are not identical to the dimensions of any small item type,
- N is either a pure or a contaminated intermediate plate, and the dimensions of N do not allow for accommodating another small item of any type, i.e., a piece of waste is obtained.

Throughout the cutting process, a 0-plate is not considered for any further cut since it corresponds to a final plate, or it is discarded. Therefore, cuts will only be performed on intermediate plates by considering the following rules:

- Cuts will be possible in parallel to the y-axis if the dimensions of an intermediate plate $N = (ox, oy, x, y)$ are not identical to the dimensions of a small item type but at least one more item type j with $l_j^s < x$ exists, which can be accommodated by N.
- Similarly, cuts will be possible in parallel to the x-axis if the dimensions of N are not identical to the dimensions of a small item type but at least one more item type j with $w_j^s < y$ exists, which can be accommodated by N.

3.2. Patterns generation

A procedure for the generation of all feasible cutting patterns for the 2D_UG_SLOPP_D can now be designed as follows:

Generate all intermediate plates as well as the corresponding 0-plate which can be obtained from all vertical and horizontal cuts performed to the original plate (large object) and decide for each particular plate, whether it can be discarded or not. For those intermediate plates which have been retained from the previous step (and, which may be pure or contaminated), one proceeds in the same way. Again, all possible cuts are generated. The obtained intermediate plates and the corresponding 0-plate are checked again with respect to whether they can be discarded or not. The procedure terminates when no more intermediate plates exist, which have to be considered for further cutting.

Now, any cutting pattern for the large object can be determined by means of the procedure as follows:

From the given large object select one and only one cut. If the selected cut corresponds to a final plate, stop. Otherwise, consider the generated division and from each of its intermediate plate select again

one and only one cut, and so on, until all visited plates are final plates. This sequence, which gives a complete path within the tree, corresponds to a cutting pattern.

We note that the value of a cutting pattern is the sum of the values of all the provided small items within the pattern. Consequently, the 2D_UG_SLOPP_D can be solved by generating all feasible cutting patterns and finding the most valuable one.

3.3. Duplicated patterns

Given an intermediate plate $N = (ox, oy, x, y)$, $x + y$-1 options exist for performing cuts on it, namely x-1 cuts in parallel to the y-axis at positions (1,…, x-1), y-1 in parallel to the x-axis at positions (1,…, y-1), and the 0-cut. Correspondingly, $x + y$-1 sub-rectangles could be obtained from N, of which a significant number would have the same dimensions. These duplications can easily be discarded from the tree-search procedure, as can be seen as follows:

Let N be pure, then application of a vertical guillotine cut at position z on the intermediate plate would yield two intermediate plates (ox, oy, z, y) and $(ox + z, oy, x - z, y)$ which are both pure, too. Obviously, two identical sub-rectangles could be obtained by performing a vertical cut to N at position x-z, which is the symmetric equivalent of a cut at position z. Thus, the set of the vertical cuts can be limited to half the length of the intermediate plate. This line of argumentation can be repeated for horizontal cuts. Note that similar techniques have been used for the standard 2D_SLOPP without defects by different authors in the literature (see, e.g., [11,7,24]).

4. The Proposed approach

4.1. Overview

A beam search is a truncated tree search approach which was first developed in the artificial intelligence community for speech recognition by Lowerre [21] and image understanding problems by Rubin [28]. Since then, several applications have been reported for solving other combinatorial optimization problems such as scheduling problems [23], circular packing problems [4], vehicle routing problems [16], assembly line balancing problems [6], and facility location problems [12].

In the beam search method, exhaustive enumeration is avoided by performing a partial search in the solution space. More precisely, on each level of the search tree, only a subset of the nodes (called the set of *elite nodes*) is selected for further branching, and the other nodes are discarded (see Fig. 1).

The elite nodes which are selected by applying *evaluation* or *filtering operators* (evaluation criteria) are those having a high potential to lead to an optimal solution of the problem. The number of elite nodes to be investigated at each level is also called the *beam width*. The beam search procedure terminates when no more nodes exist, which have to be considered for further branching (more details can be found, e.g., in [4]).

Note that in several implementations of beam search-based algorithms, a variable beam width is used where a smaller or a larger width may be needed for each level of the search process (see, e.g., [30]). Accordingly, in a variable beam search algorithm, the number of elite nodes is not necessarily constant and may change from one iteration to another (see Fig. 1). Therefore, an algorithm with a variable beam width provides a variable number of nodes in each iteration by which the risk of eliminating good nodes is reduced.

4.2. An enhanced tree-search procedure

Let $\Omega_x(L_0)$ be the set of all possible positions for (guillotine) cuts, which can be carried out to the original plate (large object) in parallel to the y-axis (vertical cuts), and $\Omega_y(W_0)$ be the set of all possible

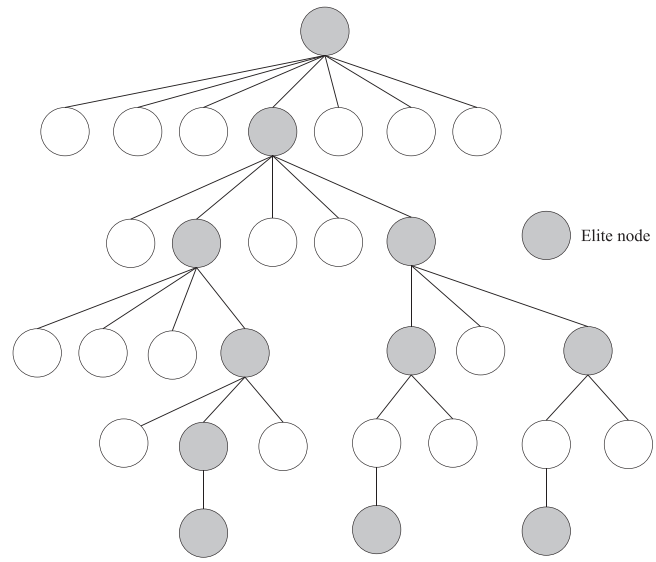

Fig. 1. The beam search tree with a variable beam width.

positions for (guillotine) cuts, which can be carried out to the original plate (large object) in parallel to the x-axis (horizontal cuts), respectively. Generate all divisions (i.e., pairs of intermediate plates resulting from all vertical and horizontal cuts) as well as the corresponding 0-plate. Decide for each division, whether it has to be considered further or whether it can be discarded. Therefore, the beam search strategy can be used by which only the most promising divisions are retained for further consideration, while the remaining ones are pruned off permanently.

Following the general procedure of the beam search given in Section 4.1, the subset of divisions which is selected by evaluation criteria is called the set of elite divisions. In addition, the number of elite divisions to be investigated is called the beam width. Note that the beam search cannot recover from a wrong decision, and it does not guarantee to find an optimal solution. Nevertheless, a wider beam width reduces the risk of eliminating a division that would ultimately lead to the optimal solution.

With respect to the elementary tree-search procedure presented in Section 3, a particular plate $N = (ox, oy, x, y)$, which can either be an intermediate plate of a division retained from the previous step or a 0-plate, is discarded if one of the following geometric conditions holds:

- N is a 0-plate, but is not a final plate, i.e., it is either a contaminated plate or its dimensions are not identical to the dimensions of any small item type.
- N is either a pure or a contaminated intermediate plate, and the dimensions of N do not allow for accommodating another small item of any type, i.e., a piece of waste is obtained.

For those intermediate plates for which none of these conditions holds (and, which may be pure or contaminated), one proceeds in the same way. Again, all possible cuts are generated. Cuts will be possible in parallel to the y-axis if the dimensions of N are not identical to the dimensions of a small item type but at least one more item type j with $l_j^s < x$ exists, which can be accommodated by N. Likewise, cuts will be possible in parallel to the x-axis if the dimensions of N are not identical to the dimensions of a small item type but at least one more item type j with $w_j^s < y$ exists, which can be accommodated by N. The obtained divisions are checked again by means of the evaluation criteria by which only the most promising divisions are retained for further consideration. Subsequently, the intermediate plates and the corresponding 0-plate are checked with respect to whether they can be discarded or not by means of the geometric conditions. The procedure terminates

when no more intermediate plates exist, which have to be considered for further cutting. Now, a cutting pattern for the large object can be determined by means of the procedure as follows:

Starting from the large object, select one and only one cut. If the selected cut corresponds to a final plate, stop. Otherwise consider the elite division and from each of its intermediate plate select again one and only one cut, and so on, until all visited plates are final plates. This sequence, which gives a complete path within the tree, corresponds to a cutting pattern. In addition, the value of a cutting pattern is the sum of the values of all the provided small items within the pattern. Consequently, the 2D_UG_SLOPP_D can be solved by finding the most valuable cutting pattern. It should be noted that this tree-search procedure can also be equipped with a parameter *depth* which represents the maximum level permitted for the depth of the search (for a more detailed description of a depth limit in a tree-search based algorithm, see, e.g., [25]). This simple strategy by using an appropriate value for the depth parameter can improve the computational efficiency of the search process.

4.3. Recursive functions and the lower/upper bounds

For the computation of the optimal value of the guillotine cutting pattern of a plate $N = (ox,oy,x,y)$, it is of importance whether N is pure or not. Accordingly, two recursive functions for the optimal objective function values are defined, one for contaminated and one for pure plates. For a contaminated plate, it will be necessary to identify the dimensions and the location of the plate on the large object. The corresponding function will be denoted by $F(ox,oy,x,y)$. For a pure plate, it will be sufficient just to note the respective dimensions. Thus, the corresponding function will be simply denoted by $F(x,y)$ which is computed by means of the recursive function of Beasley [7]. Subsequently, the recursive function for the determination of an optimal cutting pattern for a contaminated plate $N(ox,oy,x,y)$ is the one proposed by Afsharian et al. [3].

In each iteration of the beam search, the evaluation step requires upper and lower bounds on the objective function value for the generated intermediate plates. Hence, procedures which are used to determine upper bounds for both contaminated and pure plates are described in the following section.

4.3.1. Upper bounds

Let $UB(x, y)$ represent the upper bound for the objective function value of the optimal cutting pattern for a pure plate $N = (x, y)$. Supposing a relaxation of the cutting problem by considering only the constraint related to the area of the plate, the following straightforward upper bound can be defined:

$$UB(x, y) = max\left\{\sum_{j=1}^{m} v_j \cdot a_j \,\middle|\, \sum_{j=1}^{m} a_j\,(l_j^s\, w_j^s) \le x \cdot y, \quad 0 \le a_j \le \left\lfloor \frac{x}{l_j^s} \right\rfloor \cdot \left\lfloor \frac{y}{w_j^s} \right\rfloor\right\}, \tag{1}$$

where $\lfloor a \rfloor$ is the largest integer number less than or equal to a. Note that if the decision variables are restricted to be integer, the program in (1) becomes a classic bounded knapsack problem. Although such an integer program provides a tighter upper bound, but it comes at the costs of notably higher computation time. Hence, by considering the relaxation of the classic bounded knapsack problem, the upper bound can be calculated quickly for each plate in the search process. Consequently, according to the structure of the linear program in (1), the value for the upper bound can be obtained by:

$$UB(x, y) = x \cdot y \cdot max\left\{\frac{v_j}{l_j^s \cdot w_j^s} \,\middle|\, l_j^s \le x, \quad w_j^s \le y, \quad j = 1,...,m\right\}. \tag{2}$$

Now, let $UB(ox, oy, x, y)$ represent the upper bound for the objective function value of the optimal cutting pattern for a contaminated plate $N = (ox, oy, x, y)$. An upper bound for the contaminated plate can be determined by the following modification of the upper bound presented in (2).

$$UB(ox, oy, x, y) = (x \cdot y - D_{x,y}) \cdot max\left\{\frac{v_j}{l_j^s \cdot w_j^s} \,\middle|\, l_j^s \le x, \quad w_j^s \le y, \quad j = 1,...,m\right\}, \tag{3}$$

where $(x \cdot y - D_{x,y})$ represents the useable area of the corresponding contaminated plate.

Let the corresponding contaminated plate contain or overlap with the following defects: $(x_i^d, y_i^d, l_i^d, w_i^d)$, $(i = 1,...,n)$. Then $D_{x,y}$ can also be computed by the following formula:

$$D_{x,y} = \left\{\bigcup_{i=1}^{n} d_{x,y}(i) \,\middle|\, \begin{array}{l} d_{x,y}(i) = (\min\{x + ox, x_i^d + l_i^d\} - \max\{ox, x_i^d\}) \\ \qquad \times (\min\{y + oy, y_i^d + w_i^d\} - \max\{oy, y_i^d\}) \end{array}\right\}. \tag{4}$$

4.3.2. Lower bounds

Let $LB(x, y)$ represent the lower bound for the objective function value of the optimal cutting pattern for a pure plate $N = (x, y)$. The lower bound can be computed as the maximal objective function value of all homogeneous cutting patterns (i.e., those which only contain small items of a single type each) of N as follows:

$$LB(x, y) = g(x, y) = max\left(0, v_j \cdot \left\lfloor \frac{x}{l_j^s} \right\rfloor \cdot \left\lfloor \frac{y}{w_j^s} \right\rfloor \,\middle|\, l_j^s \le x, \quad w_j^s \le y, \quad j = 1,...,m\right). \tag{5}$$

Let $LB(ox, oy, x, y)$ represent the lower bound for the objective function value of the optimal cutting pattern for a contaminated plate $N = (ox, oy, x, y)$. A lower bound for the contaminated plate can be determined by a fast heuristic which allows to reduce the search space for the computation of the recursive function $F(ox, oy, x, y)$. More precisely, a lower bound can be obtained by means of a relaxation of the cutting problem in which only subsets of the vertical and horizontal discretization set of the corresponding plate (i.e. $\Omega_x^H(x)$ and $\Omega_y^H(y)$, respectively) are applied in each iteration:

$$LB(ox, oy, x, y) = \{F(ox, oy, x, y) \,|\, \Omega_x^H(x) \subseteq \Omega_x(x), \quad \Omega_y^H(y) \subseteq \Omega_y(y)\}. \tag{6}$$

As can be seen in (6), the set of all positions, which have to be considered for vertical cuts, has been reduced from $\Omega_x(x)$ to $\Omega_x^H(x)$. Likewise, the set of all positions to be considered for horizontal cuts has changed from $\Omega_y(y)$ to the set $\Omega_y^H(y)$. Obviously, these changes will reduce the computational effort for the calculation of the recursive function $F(ox, oy, x, y)$. A more precise procedure to determine $\Omega_x^H(x)$ and $\Omega_y^H(y)$ will be presented in the following sections.

4.4. Evaluation criteria

On each level of the beam search process, elite divisions are selected with respect to their respective quality, which is determined by the corresponding evaluation criteria. In the following, different criteria are proposed which can provide high-quality evaluations without requiring excessive computing times.

4.4.1. Discretization set-based criterion

Let $\Omega_x(N)$ and $\Omega_y(N)$ represent the vertical and horizontal discretization sets for a plate N respectively. When the cardinality of the sets is large, the number of divisions obtained from all vertical and horizontal cuts will be significant, and the proposed algorithms may become computationally infeasible. Hence, the number of divisions can be reduced in the search process by considering only a predetermined number of vertical and horizontal cuts in each iteration, while the remaining cuts are eliminated.

According to the definition of the discretization sets, each element of the set, which represents the position for a guillotine cut, is computed by non-negative integer combinations of the lengths (resp. widths) of both item types and defects. In addition, the smallest length (resp. width) of all item types and defects relative to the length (resp. width) of the large object determines the upper bound for the cardinality of the vertical (horizontal) discretization set. Therefore, the number of cuts of the vertical (resp. horizontal) discretization set can be reduced in the search process by a modification of the iterative procedure suggested in Beasley [7]. The procedure starts by eliminating the piece among small item types or defects with the smallest length (resp. width) until the desired size of the set is achieved. By means of this definition, only a limited number of divisions (i.e., elite divisions) obtained from the vertical (horizontal) cuts are generated in each iteration of the search process. NOE that the number of elite divisions is referred to as the beam width which can either be fixed or variable.

If N is a pure plate, the variable beam widths for the vertical and horizontal cuts are defined by $\lambda_x^p \cdot |\Omega_x(N)|$ and $\lambda_y^p \cdot |\Omega_y(N)|$, respectively, where λ_x^p and λ_y^p are parameters which have to be specified in advance from the interval [0, 1]. Consequently, the set of all positions, which have to be considered for vertical cuts, will be reduced from $\Omega_x(N)$ to $\Omega_x^{PH}(N)$, where $|\Omega_x^{PH}(N)| \leq \lambda_x^p \cdot |\Omega_x(N)|$. Likewise, the set of all positions to be considered for horizontal cuts will change from $\Omega_y(N)$ to the set $\Omega_y^{PH}(N)$, where $|\Omega_y^{PH}(N)| \leq \lambda_y^p \cdot |\Omega_y(N)|$. Subsequently, the variable beam widths for the vertical and horizontal cuts are applied for a contaminated plate N by $\lambda_x^c \cdot |\Omega_x(N)|$ and $\lambda_y^c \cdot |\Omega_y(N)|$, respectively, where λ_x^c and λ_y^c are a parameters which have to be specified in advance from the interval [0, 1]. Thus, the set of all positions which have to be considered for vertical cuts will be reduced from $\Omega_x(N)$ to $\Omega_x^{CH}(N)$ where $|\Omega_x^{CH}(N)| \leq \lambda_x^c \cdot |\Omega_x(N)|$. Likewise, the set of all positions to be considered for horizontal cuts will change from $\Omega_y(N)$ to the set $\Omega_y^{CH}(N)$ where $|\Omega_y^{CH}(N)| \leq \lambda_y^c \cdot |\Omega_y(N)|$.

4.4.2. Rank-based criterion

Let $\Omega_x(N)$ represent the vertical and $\Omega_y(N)$ represent the horizontal discretization sets for a plate N. The number of divisions which are generated by all vertical and horizontal cuts will be $|\Omega_x(N)| + |\Omega_y(N)|$. By means of the definition, a division, which is generated by cut r ($r \in \Omega_x(N) \cup \Omega_y(N)$), consists of a pair of intermediate plates N_1^r and N_2^r. Consider the homogenous cutting patterns related to N_1^r and N_2^r, respectively. Hence, the union of these two homogenous cutting patterns provides a feasible solution to the plate N. Accordingly, $LB(N_1^r) + LB(N_2^r)$ can be considered as a value for the corresponding division. When the cardinality of the discretization sets is large, the proposed algorithms may become computationally infeasible. Hence, if the divisions are sorted by decreasing value, only a predetermined number of the best divisions (i.e., elite divisions) can be selected, and the other divisions can be discarded. Consequently, the number of divisions will be reduced in the search process which allows for an improvement of the computational efficiency.

In this context, $\lfloor \theta \cdot (|\Omega_x(N)| + |\Omega_y(N)|) \rfloor$ is proposed as a variable beam width, which allows for selecting a variable number of elite divisions from one iteration to the next in the search process. Note again that θ is a parameter, which has to be specified by the user from the interval [0, 1]. $\lfloor a \rfloor$ is the largest integer number less than or equal to a. Furthermore, since a different number of elite divisions may be preferred in each iteration according to whether a plate under consideration is contaminated or pure, two different cases can be distinguished for the determination of the beam width. NOE that this implementation was motivated by the fact that different beam widths may be appropriate for contaminated and pure plates with respect to the value for the corresponding division. Accordingly, if a plate N is pure, the variable beam width is defined by $\lfloor \theta^p \cdot (|\Omega_x(N)| + |\Omega_y(N)|) \rfloor$. Otherwise, the variable beam width is applied for a contaminated plate N by means of $\lfloor \theta^c \cdot (|\Omega_x(N)| + |\Omega_x(N)|) \rfloor$. θ^p and θ^c are parameters, which have to be specified in advance by the user from the interval [0, 1].

4.4.3. Upper Bound-based criterion

Let $UB(N)$ represent an upper bound and $LB(N)$ represent a lower bound for a plate N. Suppose that a division is generated by performing a cut to this plate in an iteration of the search process. Then the two resulting intermediate plates N_1 and N_2 are generated. Therefore, if $UB(N_1) + UB(N_2) \leq LB(N)$, this division can be discarded without losing the optimal solution. Accordingly, if $UB(N_1) + UB(N_2)$ is not substantially larger than $LB(N)$, this may indicate that this division will not lead to an optimal solution. Accordingly, a division is discarded from the generated divisions for a plate N, if $\partial_u(UB(N_1) + UB(N_2)) \leq LB(N)$, where ∂_u is a parameter, which has to be specified by the user from the interval [0, 1].

With respect to whether a plate under consideration is pure or contaminated, two types of the above-described procedure can be defined. More precisely, if a plate N is pure, the procedure can be defined by $\partial_u^p(UB(N_1) + UB(N_2)) \leq LB(N)$. Otherwise, the procedure, which is applied for a contaminated plate N, can be defined by $\partial_u^c(UB(N_1) + UB(N_2)) \leq LB(N)$. ∂_u^p and ∂_u^c are parameters, which have to be specified in advance from the interval [0, 1].

4.4.4. Lower Bound-based criterion

Let again $UB(N)$ represent an upper bound and $LB(N)$ represent a lower bound for a plate N. Suppose that a division is generated by performing a cut to this plate in an iteration of the search process. Then the two intermediate plates N_1 and N_2 are generated. It should be noted that the optimal solution can be lost if we discard the division from the set of all possible divisions for plate N such that $LB(N_1) + LB(N_2) \leq LB(N)$. Nevertheless, if $LB(N_1) + LB(N_2)$ is substantially smaller than $LB(N)$, then this can be a sign that this division will not lead to the optimal solution. Thus, a division can be discarded from the set of previously generated divisions for a plate N, if $\partial_l(LB(N_1) + LB(N_2)) \leq LB(N)$, where ∂_l is a parameter, which has to be specified from the interval [0, 1].

With respect to whether a plate under consideration is contaminated or pure, two kinds of the above-described procedure can be defined. If a plate N is pure, the procedure can be applied by $\partial_l^p(LB(N_1) + LB(N_2)) \leq LB(N)$. Otherwise, the procedure, which is used for a contaminated plate N, can be defined by $\partial_l^c(LB(N_1) + LB(N_2)) \leq LB(N)$. It has to be noted that ∂_l^p and ∂_l^c are parameters, which have to be specified in advance by the user from the interval [0, 1].

5. Heuristics

5.1. Beam search Algorithm 1 (BS1)

5.1.1. Pure plates

Let N be a pure plate, and let $\Omega_x(N)$ and $\Omega_x(N)$, the sets of positions, which have to be considered for vertical and horizontal sets. According to the discretization set-based criterion, since the corresponding plate is pure, the variable beam widths for the vertical and horizontal cuts are defined by $\lambda_x^p \cdot |\Omega_x(N)|$ and $\lambda_y^p \cdot |\Omega_y(N)|$, respectively, where λ_x^p and λ_y^p are parameters, which have to be specified in advance from the interval [0, 1]. Consequently, the set of all positions, which have to be considered for vertical cuts, will be reduced from $\Omega_x(N)$ to $\Omega_x^{PH}(N)$ where $|\Omega_x^{PH}(N)| \leq \lambda_x^p \cdot |\Omega_x(N)|$. Likewise, the set of all positions to be considered for horizontal cuts will change from $\Omega_y(N)$ to the set $\Omega_y^{PH}(N)$ where $|\Omega_y^{PH}(N)| \leq \lambda_y^p \cdot |\Omega_y(N)|$. Now, the objective function value for the guillotine cutting pattern of a pure plate $N = (x, y)$, represented by $F_1^H(x, y)$, can be determined by means of algorithm #1:

Algorithm 1. Computation of $F_1^H(x, y)$.

Input : $N(x, y), \{l_1^s, ..., l_m^s\}, \{w_1^s, ..., w_m^s\}, \{v_1, ..., v_m\}, \{l_1^d, ..., l_n^d\}, \{w_1^d, ..., w_n^d\}$

Output : $F_1^H(x, y)$

begin
if the solution is already known **then**
  return $F_1^H(x, y)$
end if
if $x < l_0$ or $y < w_0$ **then**
  return 0
end if
$BV := g(x, y)$
foreach $z_x \in \Omega_x^{PH}(x)$ **do**
  if $z_x \le x/2$ **then**
    $BV := \max\{BV, F_1^H(z_x, y) + F_1^H(p(x - z_x), y)\}$
  end if
end foreach
foreach $z_y \in \Omega_y^{PH}(y)$ **do**
  if $z_y \le y/2$ **then**
    $BV := \max\{BV, F_1^H(x, z_y) + F_1^H(x, q(y - z_y))\}$
  end if
end foreach
return BV
end

5.1.2. Contaminated plates

When N is contaminated, let the sets of positions, which have to be considered for vertical and horizontal sets, be $\Omega_x(N)$ and $\Omega_x(N)$. With respect to the discretization set-based criterion, the variable beam widths for the vertical and horizontal cuts are defined by $\lambda_x^c \cdot |\Omega_x(N)|$ and $\lambda_y^c \cdot |\Omega_y(N)|$, respectively, where λ_x^c and λ_y^c are parameters which have to be specified in advance from the interval [0, 1]. Consequently, the set of all positions, which have to be considered for vertical cuts, will be reduced from $\Omega_x(N)$ to $\Omega_x^{CH}(N)$, where $|\Omega_x^{CH}(N)| \le \lambda_x^c \cdot |\Omega_x(N)|$.

Algorithm 2. Computation of $F_1^H(ox, oy, x, y)$.

Input : $N(ox, oy, x, y), \{l_1^s, ..., l_m^s\}, \{w_1^s, ..., w_m^s\}, \{v_1, ..., v_m\}, \{l_1^d, ..., l_n^d\}, \{w_1^d, ..., w_n^d\}$

Output : $F_1^H(ox, oy, x, y)$

begin
if the solution is already known **then**
  return $F_1^H(ox, oy, x, y)$
end if
if $x < l_0$ **or** $y < w_0$ **then**
  return 0
end if
if $CP(ox, oy, x, y) = 0$ **then**
  return $F_1^H(x, y)$
end if
$BV := 0$
foreach $z_x \in \Omega_x^{CH}(x)$ **do**
  $BV := \max\{BV, F_1^H(ox, oy, z_x, y) + F_1^H(ox + z_x, oy, p(x - z_x), y),$
  $F_1^H(ox, oy, p(x - z_x), y) + F_1^H(ox + x - z_x, oy, z_x, y)\}$
end foreach
foreach $z_y \in \Omega_y^{CH}(y)$ **do**
  $BV := \max\{BV, F_1^H(ox, oy, x, z_y) + F_1^H(ox, oy + z_y, x, q(y - z_y)),$
  $F_1^H(ox, oy, x, q(y - z_y)) + F_1^H(ox, oy + y - z_y, x, z_y)\}$
end foreach
return BV
end

Similarly, the set of all positions to be considered for horizontal cuts will change from $\Omega_y(N)$ to the set $\Omega_y^{CH}(N)$ where $|\Omega_y^{CH}(N)| \leq \lambda_y^c \cdot |\Omega_y(N)|$. Now the objective function value for the guillotine cutting pattern of a contaminated plate $N = (ox, oy, x, y)$, represented by $F_1^H(ox, oy, x, y)$, can be computed by means of algorithm #2. Note that in this algorithm, if the plate under consideration has been identified to be pure by means of the characteristic function $CP(ox, oy, x, y)$, algorithm #1 is used to continue the computations for the determination of the objective function value for the corresponding pure plate.

5.2. Beam search Algorithm 2 (BS2)

5.2.1. Pure plates

Throughout the search process let N be a pure plate. Let a corresponding division consist of the two intermediate plates N_1 and N_2. With respect to the upper bound-based criterion, this division is discarded from the generated divisions for the plate N, if $\partial_u^p(UB(N_1) + UB(N_2)) \leq LB(N)$. Likewise, according to the lower bound-based criterion, a division which has been retained, is checked whether it has to be discarded from the elite divisions for a plate N or not by applying the inequality $\partial_l^p(LB(N_1) + LB(N_2)) \leq LB(N)$. It has to be noted that ∂_u^p and ∂_l^p are parameters, which have to be specified in advance by the user from the interval [0, 1]. Furthermore, a depth bound, which limits the level of the search for pure plates, can also be applied. The value for the depth parameter will be depicted by $Depth(P)$. Now the objective function value for the guillotine cutting pattern of a pure plate $N = (x, y)$, represented by $F_2^H(x, y)$, can be determined by means of algorithm #3:

5.2.2. Contaminated plates

Let N be a contaminated plate. Furthermore, let a corresponding division resulting from a guillotine cut consist of the two intermediate plates N_1 and N_2. According to the upper bound-based criterion, this division is discarded from the divisions for the contaminated plate N, if $\partial_u^c(UB(N_1) + UB(N_2)) \leq LB(N)$. Likewise, with respect to the lower bound-based criterion, a division which has been retained, is checked whether it has to be discarded from the elite divisions for a plate N or not by checking the inequality $\partial_l^c(LB(N_1) + LB(N_2)) \leq LB(N)$. ∂_u^c and ∂_l^c are parameters, which have to be specified in advance from the interval [0, 1]. It has to be noted that BS1 can be applied for the determination of lower bounds for contaminated plates. Furthermore, a depth bound which limits the level of the search for contaminated plates can also be applied. The value for the depth parameter will be depicted by $Depth(C)$.

Algorithm 3. Computation of $F_2^H(x, y)$.

Input : $N(x,y), \{l_1^s,...,l_m^s\}, \{w_1^s,...,w_m^s\}, \{v_1,...,v_m\}, \{l_1^d,...,l_n^d\}, \{w_1^d,...,w_n^d\}$
Output : $F_2^H(x,y)$
begin
if the solution is already known **then**
  return $F_2^H(x,y)$
end if
if $x < l_0$ or $y < w_0$ **then**
  return 0
end if
$BV := LB(x,y)$
if $DepthCount \le Depth(p)$ **then** (default value of $DepthCount$ is zero)
  $DepthCount := DepthCount + 1$
  foreach $z_x \in \Omega_x(x)$ **do**
    if $z_x \le x/2$ **then**
      if $\partial_u^p\left(UB(z_x,y) + UB(p(x-z_x),y)\right) > BV$ **then**
        if $\partial_l^p\left(LB(z_x,y) + LB(p(x-z_x),y)\right) > LB(x,y)$ **then**
          $BV := \max\left\{BV, F_2^H(z_x,y) + F_2^H(p(x-z_x),y)\right\}$
        end if
      end if
    end if
  end foreach
  $DepthCount := DepthCount - 1$
  foreach $z_y \in \Omega_y(y)$ **do**
    if $z_y \le y/2$ **then**
      if $\partial_u^p\left(UB(x,z_y) + UB(x,q(y-z_y))\right) > BV$ **then**
        if $\partial_l^p\left(LB(x,z_y) + LB(x,q(y-z_y))\right) > LB(x,y)$ **then**
          $BV := \max\left\{BV, F_2^H(x,z_y) + F_2^H(x,q(y-z_y))\right\}$
        end if
      end if
    end if
  end foreach
  $DepthCount := DepthCount - 1$
end if
return BV
end

Now the objective function value for the guillotine cutting pattern of a contaminated plate $N = (ox, oy, x, y)$, represented by $F_2^H(ox, oy, x, y)$, can be determined by means of algorithm #4. In this algorithm, if the plate under consideration has been identified to be pure by means of the characteristic function $CP(ox, oy, x, y)$, algorithm #3 is used to continue the computations for the determination of the objective function value for the corresponding pure plate.

Algorithm 4. Computation of $F_2^H(ox, oy, x, y)$.

Input : $N(ox,oy,x,y), \{l_1^s,...,l_m^s\}, \{w_1^s,...,w_m^s\}, \{v_1,...,v_m\}, \{l_1^d,...,l_n^d\}, \{w_1^d,...,w_n^d\}$
Output : $F_2^H(ox,oy,x,y)$
begin
if the solution is already known **then**
  return $F_2^H(ox,oy,x,y)$
end if
if $x < l_0$ or $y < w_0$ **then**
  return 0
end if
if $CP(ox,oy,x,y) = 0$ **then**
  return $F_2^H(x,y)$
end if
$BV := LB(ox,oy,x,y)$
if $DepthCount \le Depth(C)$ **then** (default value of $DepthCount$ is zero)
  $DepthCount := DepthCount + 1$
  foreach $z_x \in \Omega_x(x)$ **do**
    if $\partial_u^c\left(UB(ox,oy,z_x,y) + UB(ox+z_x,oy,p(x-z_x),y)\right) > BV$ **then**
      if $\partial_l^c\left(LB(ox,oy,z_x,y) + LB(ox+z_x,oy,p(x-z_x),y)\right) > LB(ox,oy,x,y)$ **then**
        $BV := \max\left\{BV,\ F_2^H(ox,oy,z_x,y) + F_2^H(ox+z_x,oy,p(x-z_x),y)\right\}$
      end if
    end if
    if $\partial_u^c\left(UB(ox,oy,p(x-z_x),y) + UB(ox+x-z_x,oy,z_x,y)\right) > BV$ **then**
      if $\partial_l^c\left(LB(ox,oy,p(x-z_x),y) + LB(ox+x-z_x,oy,z_x,y)\right) > LB(ox,oy,x,y)$ **then**
        $BV := \max\left\{BV,\ F_2^H(ox,oy,p(x-z_x),y) + F_2^H(ox+x-z_x,oy,z_x,y)\right\}$
      end if
    end if
  end foreach
  $DepthCount := DepthCount - 1$
  foreach $z_y \in \Omega_y(y)$ **do**
    if $\partial_u^c\left(UB(ox,oy,x,z_y) + UB(ox,oy+z_y,x,q(y-z_y))\right) > BV$ **then**
      if $\partial_l^c\left(LB(ox,oy,x,z_y) + LB(ox,oy+z_y,x,q(y-z_y))\right) > LB(ox,oy,x,y)$ **then**
        $BV := \max\left\{BV,\ F_2^H(ox,oy,x,z_y) + F_2^H(ox,oy+z_y,x,q(y-z_y))\right\}$
      end if
    end if
    if $\partial_u^c\left(UB(ox,oy,x,q(y-z_y)) + UB(ox,oy+y-z_y,x,z_y)\right) > BV$ **then**
      if $\partial_l^c\left(LB(ox,oy,x,q(y-z_y)) + LB(ox,oy+y-z_y,x,z_y)\right) > LB(ox,oy,x,y)$ **then**
        $BV := \max\left\{BV,\ F_2^H(ox,oy,x,q(y-z_y)) + F_2^H(ox,oy+y-z_y,x,z_y)\right\}$
      end if
    end if
  end foreach
  $DepthCount := DepthCount - 1$
end if
return BV
end

5.3. Beam search Algorithm 3 (BS3)

5.3.1. Pure plates

Let N be a pure plate, and let $\Omega_x(N)$ and $\Omega_x(N)$, the sets of positions, which have to be considered for vertical and horizontal sets. The number of divisions, which are generated by all vertical and horizontal cuts, will be $|\Omega_x(N)| + |\Omega_y(N)|$. Let the corresponding divisions be sorted by decreasing values in which each value is determined by $LB\ (N_1^r) + LB\ (N_2^r)$. In addition, the variable beam width is defined by $\lfloor \theta^p \cdot (\ |\Omega_x(N)| + |\Omega_y(N)|\ ) \rfloor$, where θ^p is a parameter, which has to be

specified in advance by the user from the interval [0, 1]. By means of the definition of the rank-based criterion, only a predetermined number of the best divisions is selected, and the other divisions are discarded. It should be noted that a depth bound, which limits the level of the search for pure plates can also be applied. The value for the depth parameter will be depicted by $Depth(P)$. Now the objective function value for the guillotine cutting pattern of a pure plate $N = (x, y)$, represented by $F_3^H(x, y)$, can be computed by means of algorithm #5.

Algorithm 5. Computation of $F_3^H(x, y)$.

5.3.2. Contaminated plates

When N is a contaminated plate, let $\Omega_x(N)$ and $\Omega_y(N)$ be the sets of positions, which have to be considered for vertical and horizontal cuts. Accordingly, the number of divisions, which are generated by all vertical and horizontal cuts, will be $|\Omega_x(N)| + |\Omega_y(N)|$. Let the corresponding divisions be sorted by decreasing values in which each value is determined by $LB(N_1^r) + LB(N_2^r)$. In addition, the variable beam width is defined by $\lfloor \theta^c \cdot (\,|\Omega_x(N)| + |\Omega_y(N)|\,) \rfloor$, where θ^c is a parameter, which has to be specified in advance from the interval [0, 1].

Input : $N(x,y), \{l_1^s,...,l_m^s\}, \{w_1^s,...,w_m^s\}, \{v_1,...,v_m\}, \{l_1^d,...,l_n^d\}, \{w_1^d,...,w_n^d\}$
Output : $F_3^H(x,y)$
begin
if the solution is already known **then**
  return $F_3^H(x,y)$
end if
if $x < l_0$ or $y < w_0$ **then**
  return 0
end if
$BV := LB(x,y)$
if $DepthCount \le Depth(p)$ **then** (default value of $DepthCount$ is zero)
  $DepthCount := DepthCount + 1$
  foreach $z_x \in List(x,y)$ **do**
    if $z_x \le x/2$ **then**
      $BV := \max\{BV, F_3^H(z_x, y) + F_3^H(p(x - z_x), y)\}$
    end if
  end foreach
  $DepthCount := DepthCount - 1$
  foreach $z_y \in List(x,y)$ **do**
    if $z_y \le y/2$ **then**
      $BV := \max\{BV, F_3^H(x, z_y) + F_3^H(x, q(y - z_y))\}$
    end if
  end foreach
  $DepthCount := DepthCount - 1$
end if
return BV
end

Algorithm 6. Computation of $F_3^H(ox, oy, x, y)$.

Input : $N(ox, oy, x, y), \{l_1^s, ..., l_m^s\}, \{w_1^s, ..., w_m^s\}, \{v_1, ..., v_m\}, \{l_1^d, ..., l_n^d\}, \{w_1^d, ..., w_n^d\}$

Output : $F_3^H(ox, oy, x, y)$

begin
if the solution is already known **then**
  return $F_3^H(ox, oy, x, y)$
end if
if $x < l_0$ or $y < w_0$ **then**
  return 0
end if
if $CP(ox, oy, x, y) = 0$ **then**
  return $F_3^H(x, y)$
end if
$BV := LB(ox, oy, x, y)$
if $DepthCount \le Depth(C)$ **then** (default value of $DepthCount$ is zero)
  $DepthCount := DepthCount + 1$
  foreach $z_x \in List(ox, oy, x, y)$ **do**
    $BV := \max\{BV, F_3^H(ox, oy, z_x, y) + F_3^H(ox + z_x, oy, p(x - z_x), y),$
    $F_3^H(ox, oy, p(x - z_x), y) + F_3^H(ox + x - z_x, oy, z_x, y)\}$
  end foreach
  $DepthCount := DepthCount - 1$
  foreach $z_y \in List(ox, oy, x, y)$ **do**
    $BV := \max\{BV, F_3^H(ox, oy, x, z_y) + F_3^H(ox, oy + z_y, x, q(y - z_y)),$
    $F_3^H(ox, oy, x, q(y - z_y)) + F_3^H(ox, oy + y - z_y, x, z_y)\}$
  end foreach
  $DepthCount := DepthCount - 1$
end if
return BV
end

By means of the definition of the rank-based criterion, only a predetermined number of the best divisions is selected, and the other divisions are discarded. It has to be noted that BS1 can be applied for the determination of lower bounds for contaminated plates. Furthermore, a depth bound, which limits the level of the search for contaminated plates, can also be applied. The value for the depth parameter will be depicted by $Depth(C)$. Now the objective function value for the guillotine cutting pattern of a contaminated plate $N = (ox, oy, x, y)$, represented by $F_3^H(ox, oy, x, y)$, can be determined by means of algorithm #6. It has to be noted that in this algorithm if the plate under consideration is identified to be pure by means of the characteristic function $CP(ox, oy, x, y)$, algorithm #5 is used to continue the computations for the determination of the objective function value for the corresponding pure plate.

6. Simulation study

6.1. Design of the experiments

A set of randomly generated instances of the 2D_UG_SLOPP_D is obtained by specifying of the following problem descriptors:

- dimensions of the large object,
- number of small item types,
- value of the small item types,
- dimensions of the small item types,
- number of defects,
- dimensions of the defects,
- location of the defects.

With respect to the size of the large object, we distinguish between three categories of problem instances. In category #1, the size of the large object is fixed to 5625 square units (sq. u.), in category #2 to 22,500 square units (sq. u.), and in category #3 to 90,000 square units (sq. u.). In addition, two different shapes of the large object are considered for each category: (a) quadratic (75, 75) and (b) non-quadratic (112, 50) for category #1, (c) quadratic (150, 150) and (d) non-quadratic (225, 100) for category #2, and (e) quadratic (300, 300) and (f) non-quadratic (450, 200) for category #3.

The number of small item types (m), is set to 5, 10, 15, 20, and 25 in all the categories. The generated set of test problem instances belongs to the unweighted version of the problem. Hence, the values (in monetary units) of the small item types are equal to their area (in sq. u.). The length and the width of the small item types are picked uniformly from

the interval $[L_0/\rho^s,\ \ 3L_0/4]$ and $[W_0/\rho^s,\ \ 3W_0/4]$, respectively, in which ρ^s is set to 6, 8 and 10 in all the categories.

The number of defects (n) is set to 1, 2, 3, and 4 in all the categories. To define the dimensions of the defects, the length and the width of the defects are picked uniformly from the interval $[L_0/10,\ \ L_0/6]$ and $[W_0/10,\ \ W_0/6]$, respectively. The location of each defect, represented by the position of the lower left corner of the defect on the large object, is generated randomly using a uniform distribution in the intervals $[0,\ \ L_0 - l^d]$ and $[0,\ \ W_0 - w^d]$. These values are then rounded mathematically to obtain integer numbers. Note that for the generation of the defects we avoid overlapping to allow for a better numerical manageability of the number of defects per instance.

The above descriptors include three parameter values for the size of the large object, two for the shape of the large object, five for the number of small item types, and three for the size of the small item types. This gives rise to $(3 \times 2 \times 5 \times 3 =)$ 90 classes of problem instances. For each of these classes, 15 instances are generated which are considered with one to four defects (n = 1, 2, 3 and 4). Thus, in total $(90 \times 15 \times 4 =)$ 5400 problem instances are included in the numerical experiments.

6.2. Parameters setting for the heuristics

To determine appropriate values for the parameters, which are used by each heuristic algorithm, a series of preliminary tests has been performed on the randomly generated instances. It has been found that there is a trade-off between solution quality and computation time. More precisely, the objective function value is improved with an increase of the value of the corresponding parameters (i.e., by means of wider beam widths) at the cost of increased computational times. Accordingly, the parameter values that showed to provide the reasonable trade-offs between solution quality and computation time have been chosen. In the following, for each heuristic algorithm, the corresponding parameters are introduced.

6.2.1. Beam search Algorithm 1 (BS1)

With respect to the structure of the first heuristic algorithm, BS1, the values of the parameters λ_x^p and λ_y^p have both been set to 1, i.e. $\lambda_x^p = \lambda_y^p = 1$, for performing on the data set. This allows for considering all the elements of the discretization sets, correspondingly all the generated divisions, which are determined for pure plates. In contrast to that, the values of the parameters λ_x^c and λ_y^c for contaminated plates have both been set to 0.85, i.e. $\lambda_x^c = \lambda_y^c = 0.85$. By means of the discretization set-based criterion, only 85% of divisions (i.e. elite divisions) obtained from the vertical and horizontal cuts are generated in each iteration of the search process.

6.2.2. Beam search Algorithm 2 (BS2)

The second heuristic algorithm uses the upper bound- and lower bound-based criteria in the search process. It can also be equipped with the depth bound. The values of the parameter ∂_u^p and ∂_l^p, related to pure plates, have been fixed at 1 and 0, respectively, i.e. $\partial_u^p = 1$ and $\partial_l^p = 0$. However, for contaminated plates, the values of the parameters ∂_u^c and ∂_l^c have both been set to 0.99, i.e. $\partial_u^c = \partial_l^c = 0.99$. It has to be noted that, by means of these parameters, in each iteration of the algorithm, all divisions, which may lead to an optimal solution, are kept for further branching while the remaining divisions are eliminated. An unrestricted depth is considered for pure plates, i.e. $Depth(P) = \infty$. In contrast to that, a depth bound, which limits the level of the search for contaminated plates, is set to 2 i.e. $Depth(C) = 2$. Moreover, for the computation of the lower bounds for contaminated plates, BS1 with the values of the parameters $\lambda_x^p = \lambda_y^p = 1$ and $\lambda_x^c = \lambda_y^c = 0.75$ is employed.

6.2.3. Beam search Algorithm 3 (BS3)

BS3 relies on both depth bound and rank-based criterion in the search process. Accordingly, a value of one has been chosen for the parameter θ^p, i.e. $\theta^p = 1$. This allows for considering all the divisions generated from pure plates in each iteration of the algorithm. In contrast, the value of the parameter θ^c for contaminated plates has been set to 0.30, i.e. $\theta^c = 0.30$. By means of the rank-based criterion, only 30% of the divisions (i.e. elite divisions) are selected, and the other divisions are discarded in the research process. In addition, for pure plates, no limit to the search depth is imposed, i.e. $Depth(P) = \infty$. However, a depth bound, which limits the level of the search for contaminated plates, is set to 2, i.e. $Depth(C) = 2$. Moreover, according to the structure of the proposed algorithm, for the computation of the lower bounds for contaminated plates, BS1 is applied with the parameter's values $\lambda_x^p = \lambda_y^p = 1$ and $\lambda_x^c = \lambda_y^c = 0.75$.

6.3. Performance measures

The choice of performance measures for the numerical experiments, to evaluate the proposed algorithms, involves both solution quality and computation time. The solution quality is evaluated by the average approximation ratio (AAR), the average deviation (in percent) of the objective function values (DOV) and the percentage of instances with a zero deviation (IZD). The computational time ratio (CTR) is also determined in order to evaluate the performance of the proposed algorithms with respect to the running times. In the following, these performance measures are defined:

The average approximation ratio (AAR) is determined for a selected group of instances by considering Z_H/Z_B, where Z_H denotes the average objective function values (per instance) obtained by the corresponding beam search algorithm and Z_B denotes the best (on average) objective function values (per instance) provided by the dynamic programming algorithm (DP) proposed by Afsharian et al. [3]. By means of these definitions, the average deviation of the objective function values (DOV) is also computed as a percentage: $(Z_B - Z_H) \times 100/Z_B$. In addition, IZD represents the percentage of instances with a zero deviation. The computational time ratio (CTR) is measured as a percentage for a selected group of instances by applying $(T_H/T_B) \times 100$, where T_H represents the average computing time obtained by the corresponding beam search algorithm and T_B denotes the average computing time obtained by the DP algorithm.

6.4. Implementation and hardware

The proposed algorithms have been encoded in C#, Version 4.0. All experiments have been carried out on a PC with an AMD E-350 @ processor with 4 GB of RAM and a clock speed of 1.60 GHz under the Windows Operating System.

6.5. Results of the heuristic algorithms

In this section, the performance of the proposed heuristic algorithms is evaluated, and the results are compared to those obtained by the approach of Afsharian et al. [3]. In the following, (as also introduced in the previous sections) the dynamic programming algorithm of Afsharian et al. [3] is denoted by (DP), while the proposed beam search algorithms are shown by BS algorithms. Tables 1, 2, and 3 inform about the performance of the proposed algorithms, BS1, BS2 and BS3 with respect to the different number of item types in all categories.

As can be seen in Table 1, (i.e., results obtained by BS1), the percentage of instances with a zero deviation (IZD) decreases as a number of item types grows. It decreases from 68.61% (m=5) to 40.83% (m=25) in category #1, from 68.89% (m=5) to 45.83% (m=25) in category #2, and from 72.78% (m=5) to 47.22% (m=25) in category #3. Considering IZD in Tables 2 and 3, similar trends can also be observed for BS2 and BS3. The reason can be explained by the fact that with more types of small items, the number of cuts, which have to be performed on the large object, increases. Subsequently, the number of necessary divisions, which have to be considered for finding an optimal solution, also increases.

Table 1
Results obtained by BS1.

	number of item types (m)	DOV [%]	IZD [%]	average computing time [sec]		CTR [%]
				DP	BS1	
category 1	5	3.22	68.61	2.942	0.297	10.10
	10	1.74	65.83	4.724	0.642	13.59
	15	1.26	53.05	5.640	0.827	14.66
	20	1.17	51.94	6.968	1.303	18.70
	25	1.77	40.83	8.050	1.693	21.03
	Av.	1.83	56.06	5.665	0.952	16.80
category 2	5	3.49	68.89	25.981	2.567	9.88
	10	1.12	62.22	47.438	6.259	13.19
	15	0.97	61.11	63.498	8.288	13.05
	20	0.99	51.94	85.171	13.689	16.07
	25	1.16	45.83	107.368	20.001	18.63
	Av.	1.55	58.00	65.891	10.161	14.53
category 3	5	3.02	72.78	128.021	11.986	9.36
	10	1.12	67.50	323.847	38.721	11.96
	15	0.88	63.33	548.639	66.728	12.16
	20	0.81	58.61	884.400	123.739	13.99
	25	0.94	47.22	1184.630	169.320	14.29
	Av.	1.36	61.89	613.907	82.099	13.37

Table 2
Results obtained by BS2.

	number of item types (m)	DOV [%]	IZD [%]	average computing time [sec]		CTR [%]
				DP	BS2	
category 1	5	1.03	80.27	2.942	0.329	11.18
	10	1.04	66.66	4.724	0.513	10.86
	15	1.33	46.38	5.640	0.749	13.28
	20	1.66	32.50	6.968	0.939	13.47
	25	2.26	27.50	8.050	1.316	16.34
	Av.	1.46	50.67	5.665	0.769	13.57
category 2	5	1.31	80.27	25.981	2.112	8.13
	10	0.97	64.72	47.438	3.699	7.79
	15	0.96	53.33	63.498	5.334	8.40
	20	1.24	36.11	85.171	7.981	9.37
	25	1.45	30.00	107.368	11.406	10.62
	Av.	1.19	52.89	65.891	6.106	9.26
category 3	5	0.91	81.66	128.021	9.773	7.63
	10	0.66	70.83	323.847	21.384	6.60
	15	0.83	58.88	548.639	35.219	6.42
	20	1.06	47.50	884.400	60.313	6.82
	25	1.08	38.61	1184.630	82.795	6.99
	Av.	0.91	59.50	613.907	41.897	6.82

Table 3
Results obtained by BS3.

	number of item types (m)	DOV [%]	IZD [%]	average computing time [sec]		CTR [%]
				DP	BS3	
category 1	5	1.27	86.11	2.942	0.249	8.46
	10	1.07	69.44	4.724	0.403	8.53
	15	1.19	48.33	5.640	0.539	9.56
	20	1.52	35.83	6.968	0.734	10.53
	25	2.14	29.17	8.050	0.997	12.38
	Av.	1.44	53.78	5.665	0.584	10.31
category 2	5	1.82	80.56	25.981	1.706	6.56
	10	1.05	64.44	47.438	2.918	6.15
	15	0.93	55.83	63.498	4.080	6.42
	20	1.26	37.22	85.171	6.158	7.23
	25	1.47	30.83	107.368	8.894	8.28
	Av.	1.31	53.77	65.891	4.751	7.21
category 3	5	1.48	81.39	128.021	8.181	6.39
	10	0.69	71.11	323.847	18.068	5.58
	15	0.91	59.17	548.639	29.071	5.29
	20	0.95	51.11	884.400	49.181	5.56
	25	1.05	38.89	1184.630	68.658	5.79
	Av.	1.02	60.33	613.907	34.632	5.64

Table 4
Comparative results.

	category #	DOV [%]	IZD [%]	average computing time [sec]		CTR [%]
				DP algorithm	BS algorithm	
BS1	1	1.83	56.06	5.665	0.952	16.80
	2	1.55	58.00	65.891	10.161	14.53
	3	1.36	61.89	613.907	82.099	13.37
BS2	1	1.46	50.67	5.665	0.769	13.57
	2	1.19	52.88	65.891	6.106	9.26
	3	0.91	59.50	613.907	41.897	6.82
BS3	1	1.44	53.78	5.665	0.584	10.31
	2	1.31	53.77	65.891	4.751	7.21
	3	1.02	60.33	613.907	34.632	5.64

Accordingly, by applying the corresponding evaluation criteria the probability that these kinds of divisions are discarded from the search process increases with an increasing number of small item types.

From Table 2 it can be taken that the percentage of instances with a zero deviation (IZD) grows with an increase in the size of the large object. On average, it increases from 50.67% in category #1–52.89% in category #2 and to 59.50% in category #3. Furthermore, the average deviation of the objective function value (DOV) decreases as the size of the large object increases. On average, it decreases from 1.46% in category #1 to 1.19% in category #2, and then to 0.91% in category #3. The same results can also be seen in Tables 1 and 3, which present the values of these performance measures over the categories for BS1 and BS3, respectively. The reason for this behaviour can be explained by the fact that with an increasing size of the large object, the number of divisions also increases. Thus, the risk of eliminating a division that would ultimately lead to an optimal solution is reduced.

Table 3 shows that the required average computing time of BS3 in category #1 represents at most 12.38% of the time required by the DP algorithm. The maximum amount of the computational time ratio (CTR) for categories #2 and #3 are also equal to 8.28% and 6.39%, respectively. In addition, the required average computing time of the BS3 algorithm increases with an increase in the number of item types in each category. Similar trends can be seen in Tables 1 and 2 for BS1 and BS2, respectively. The reason is that the number of elite divisions, which have to be considered in the search process, is affected significantly by the number of item types. Hence, the problem becomes more difficult to solve with a growing number of small item types. Moreover, the average computing times increase much faster for the problem instances in category #2 (i.e., when the original plate is larger) than for those in category #1. Similar observations can also be made by comparing the results of category #2 and category #3. Hence, the results verify that the absolute dimensions of the large object affect the running time of the proposed algorithms.

6.6. Performance comparison of the proposed algorithms

This section compares the results obtained by the proposed algorithms to compare the solution quality, the percentage of instances with a zero deviation (IZD) and the average deviation (in percent) of the objective function value (DOV) are compared in each category. Moreover, the average computing times obtained by DP and the proposed algorithms are compared. The comparative results are summarized in Table 4.

As can be seen in Table 4, the average computing times obtained by the DP algorithm in categories #1, #2 and #3 are 5.665 s, 69.891 s, and 613.907 s, respectively. Hence, among the three heuristic algorithms, BS1 is computationally quite demanding, and can only be used for small or medium size instances in categories #1 and #2, i.e. the average computing time obtained by BS1 in category #3 is equal to 82.099 s. This becomes even more apparent by considering the computing time ratios (the last column) in the three different categories.

With respect to the solution quality, the amounts of the IZD (the percentage of instances with a zero deviation) for BS1 are equal to 56.06% in category #1, 58.00% in category #2%, and 61.89% in category #3, which are obviously superior to the others. However, the DOV values of BS1 are equal to 1.83%, 1.55%, and 1.36% in category #1, #2, and #3, respectively, which are considerably higher than the DOV values of the other algorithms. Consequently, with respect to the average computing time and the average deviation of the objective function value, BS2 and BS3 outperform BS1.

Concerning the average deviation of the objective function value, the best results are given by BS2, followed by BS3 and BS1. This does not seem surprising, since BS2 and BS3 perform a detailed evaluation for all divisions in the search process, while BS1 uses only the discretization set-based criterion which is a simpler evaluation criterion. Consequently, BS1 cannot outperform the performance of BS2 and BS3 in this context.

BS3 uses the rank-based criterion in the evaluation step of the algorithm. In fact, this criterion can eliminate poor divisions, which may be quite inferior to an optimal solution. Therefore, the elimination of these divisions can improve the performance and lead to better results than those that would be obtained if only the discretization set-based criterion is applied. This can be verified by comparing the results of the DOVs of BS1 and BS2 in all categories.

BS2 also gives better results than BS1 or BS3 since it performs a more accurate detailed evaluation than either BS1 or BS3, i.e. it uses both the lower bound- and upper bound-based criteria in the detailed evaluation step. The DOV values of BS2 are lower than DOV values obtained by BS1 and BS3. Therefore, the superior performance of BS2 when compared with BS1 and BS3 can be explained by the fact that BS2 applies a two-stage evaluation by which at each iteration of the algorithm the divisions that pass the upper bound-based criterion is then evaluated by the lower bound-based criterion.

As can be seen in Table 4, for BS1, BS2 and BS3, the solution quality determined by the DOV decreases with an increasing size of the large object, i.e. the DOV decreases over the categories. Similar results can be observed for the IZD over the categories. However, only for BS2 and BS3, the relative improvement in the DOV values over the categories shows a positive growth with an increasing size of the large object. This improvement is quite large (over 22% for BS2 and over 10% for BS3) for moves from category #1 to category #2. It also shows a substantial improvement (over 30% for BS2 and over 28% for BS3) for moves from category #2 to category #3. Moreover, the average computing time decreases as the size of the large object increases for all proposed algorithms. Consequently, once again, this verifies that BS1 can only be used for small or medium size instances. In contrast to that, however, BS2 and BS3 are superior alternatives, since they do not only provide better results, but are also faster and can be applied to larger instances.

7. Conclusion and outlook on future research

In the context of sustainability in supply chain management, cutting and packing problems are of critical importance. They strive to optimize production and logistics processes to increase resource efficiency and reduce

waste. The two-dimensional cutting problem, as an important sub-problem, involves cutting small rectangular items from a large rectangular stock sheet made of materials such as glass, leather, paper, steel, textiles, or wooded board. In this paper, a beam search-based approach has been proposed to address a particular two-dimensional cutting problem with widespread applications, characterized by a large rectangular stock sheet with multiple defects. The proposed algorithms include variable beam widths with several heuristic rules, which reduce the computation times significantly, but still lead to good solutions. The performance of the proposed algorithms has been evaluated through a comprehensive simulation study. The experiments indicate that the proposed algorithms can overcome efficiently some structural and computational limitations for solving problem instances of realistic sizes.

A promising direction for future research is to broaden the scope of the proposed approach to encompass other forms of two-dimensional cutting problems with defects, including demand-constrained scenarios. This would involve investigating innovative modifications for incorporating demand constraints into the cutting process and evaluating the impact of such modifications on solution quality and computational efficiency. Although, the proposed beam search heuristic algorithms can be seen as part of the larger field of metaheuristics, preserving the fundamental elements of the approach and incorporating other metaheuristics such as tabu search or genetic algorithms might be subjects for future research. Future research can be concentrated on providing beam search heuristic algorithms where the shape of the large object or small item types is not necessarily rectangular. In the current approach, the large object is assumed to contain several defective regions, each of which is approximated by a rectangle or a set of non-overlapping rectangles of integer dimensions. However, there is potential for future research to explore more efficient representations of the defective regions. One example could be to approximate the defects using the smallest convex shape, regardless of the shape of the small items being cut.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The context of this paper builds on the material of the PhD thesis of Mohsen Afsharian [2].

References

[1] R. Aboudi, P. Barcia, Determining cutting stock patternswhen defects are present, Ann. Oper. Res. 82 (1998) 343–354.
[2] Afsharian, M. (2013). The two-dimensional, rectangular, guillotineable-layout cutting problem with defects. Shaker.
[3] M. Afsharian, A. Niknejad, G. Wäscher, A heuristic, dynamic programming-based approach for a two-dimensional cutting problem with defects, OR Spectr. 36 (4) (2014) 971–999.
[4] H. Akeb, M. Hifi, R. M'Hallah, A beam search algorithm for the circular packing problem, Comput. Oper. Res. 36 (5) (2009) 1513–1528.
[5] R. Baldacci, M.A. Boschetti, M. Ganovelli, V. Maniezzo, Algorithms for nesting with defects, Discret. Appl. Math. 163 (2014) 17–33.
[6] J. Bautista, J. Pereira, A dynamic programming based heuristic for the assembly line balancing problem, Eur. J. Oper. Res. 194 (3) (2009) 787–794.
[7] J.E. Beasley, Algorithms for unconstrained two-dimensional guillotine cutting, J. Oper. Res. Soc. 36 (4) (1985) 297–306.
[8] J.E. Beasley, An exact two-dimensional non-guillotine cutting tree search procedure, Oper. Res. 33 (1) (1985) 49–64.
[9] J.E. Beasley, A population heuristic for constrained two-dimensional non-guillotine cutting, Eur. J. Oper. Res. 156 (3) (2004) 601–627.
[10] C. Carnieri, G.A. Mendoza, W.G. Luppold, Optimal cutting of dimension parts from lumber with a defect: a heuristic solution procedure, For. Prod. J. 43 (9) (1993) 66.
[11] N. Christofides, C. Whitlock, An algorithm for two-dimensional cutting problems, Oper. Res. 25 (1) (1977) 30–44.
[12] P. Corcoran, A. Gagarin, Heuristics for k-domination models of facility location problems in street networks, Comput. Oper. Res. 133 (2021) 105368.
[13] P.C. Gilmore, R.E. Gomory, A linear programming approach to the cutting-stock problem, Oper. Res. 9 (6) (1961) 849–859.
[14] P.C. Gilmore, R.E. Gomory, Multistage cutting stock problems of two and more dimensions, Oper. Res. 13 (1) (1965) 94–120.
[15] J.F. Gonçalves, G. Wäscher, A MIP model and a biased random-key genetic algorithm based approach for a two-dimensional cutting problem with defects, Eur. J. Oper. Res. 286 (3) (2020) 867–882.
[16] J. Gromicho, J.J. van Hoorn, A.L. Kok, J.M. Schutten, Restricted dynamic programming: a flexible framework for solving realistic VRPs, Comput. Oper. Res. 39 (5) (2012) 902–909.
[17] S.G. Hahn, On the optimal cutting of defective sheets, Oper. Res. 16 (6) (1968) 1100–1114.
[18] J.C. Herz, Recursive computational procedure for two-dimensional stock cutting, IBM Journal of Research and Development 16 (5) (1972) 462–469.
[19] M. Iori, V.L. De Lima, S. Martello, F.K. Miyazawa, M. Monaci, Exact solution techniques for two-dimensional cutting and packing, Eur. J. Oper. Res. 289 (2) (2021) 399–415.
[20] H. Kellerer, U. Pferschy, D. Pisinger, Knapsack Problems, Springer, Berlin, 2004.
[21] B.T. Lowerre, The Harpy Speech Recognition System (PhD Thesis), Carnegie-Mellon University, 1976.
[22] M. Martin, P.H. Hokama, R. Morabito, P. Munari, The constrained two-dimensional guillotine cutting problem with defects: an ILP formulation, a Benders decomposition and a CP-based algorithm, Int. J. Prod. Res. 58 (9) (2020) 2712–2729.
[23] P.R. McMullen, P. Tarasewich, A beam search heuristic method for mixed-model scheduling with setups, Int. J. Prod. Econ. 96 (2) (2005) 273–283.
[24] R. Morabito, M.N. Arenales, Staged and constrained two-dimensional guillotine cutting problems: an AND/OR-graph approach, Eur. J. Oper. Res. 94 (3) (1996) 548–560.
[25] R.N. Morabito, M.N. Arenales, V.F. Arcaro, An and or graph approach for two-dimensional cutting problems, Eur. J. Oper. Res. 58 (2) (1992) 263–271.
[26] Neidlein, V., Vianna, A.C., Arenales, M.N., Wäscher, G. (2008). The two-dimensional, rectangular, guillotineable-layout cutting problem with a single defect. Working Paper Series.
[27] M. Rönnqvist, E. Åstrand, Integrated defect detection and optimization for cross cutting of wooden boards, Eur. J. Oper. Res. 108 (3) (1998) 490–508.
[28] S. Rubin, The ARGOS Image Understanding System (PhD Thesis), Carnegie-Mellon University, 1978.
[29] U. Twisselmann, Cutting rectangles avoiding rectangular defects, Appl. Math. Lett. 12 (6) (1999) 135–138.
[30] J.M. Valente, R.A. Alves, Beam search algorithms for the single machine total weighted tardiness scheduling problem with sequence-dependent setups, Comput. Oper. Res. 35 (7) (2008) 2388–2405.
[31] A.C.G. Vianna, M.N. Arenales, O problema de corte de placas defeituosas, Pesqui. Oper. 26 (2006) 185–202.
[32] G. Wäscher, H. Haußner, H. Schumann, An improved typology of cutting and packing problems, Eur. J. Oper. Res. 183 (3) (2007) 1109–1130.
[33] H. Zhang, S. Yao, Q. Liu, L. Wei, L. Lin, J. Leng, An exact approach for the constrained two-dimensional guillotine cutting problem with defects, Int. J. Prod. Res. (2022) 1–18.